



Desenvolvimento Web

Profª Sabrina B. Moreira



JavaScript

Aula 15.1: Introdução ao JavaScript

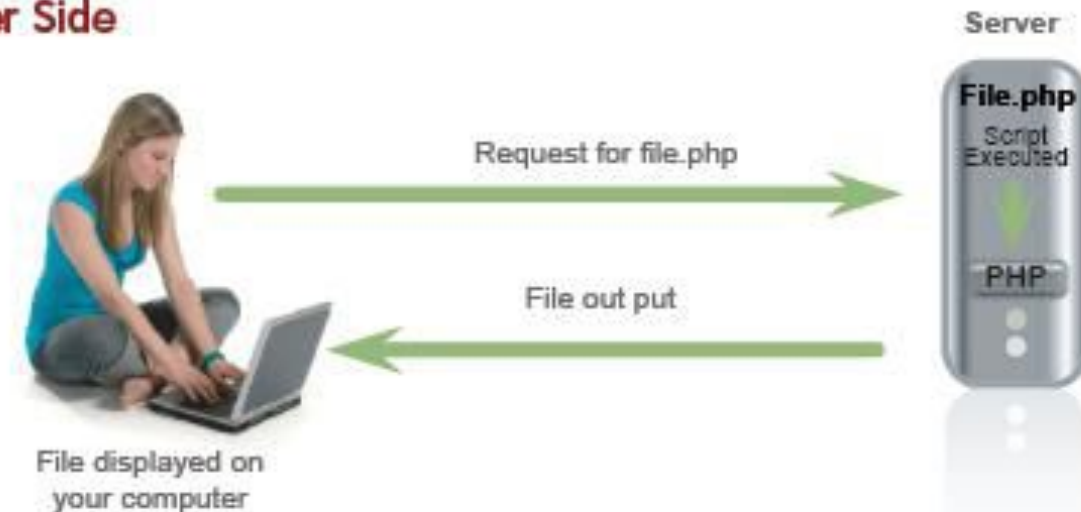
Linguagens de programação

JS: linguagens para desenvolvimento web

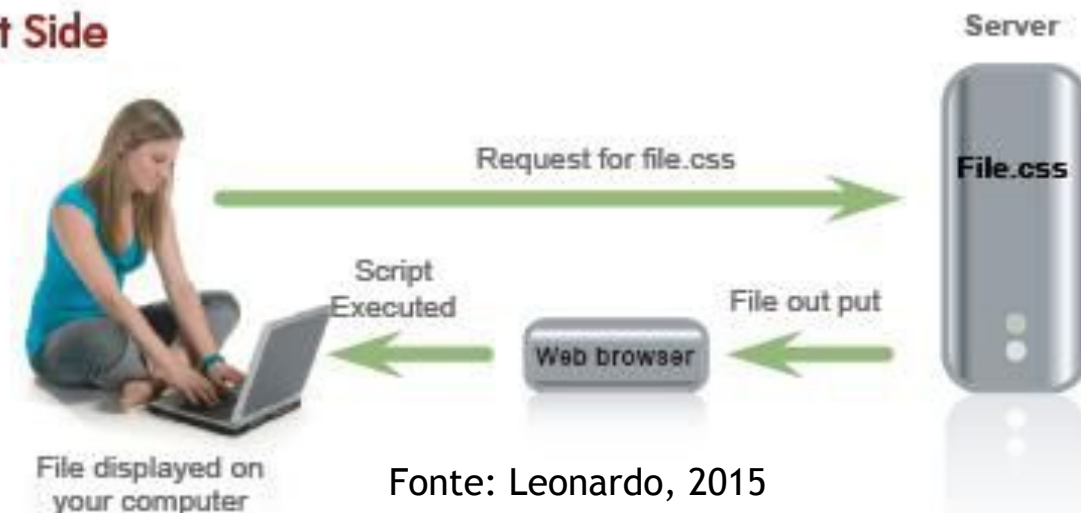
❖ As linguagens de programação para desenvolvimento web podem ser divididas em duas grandes categorias:

❖ As que rodam no servidor e as que rodam no cliente.

Server Side



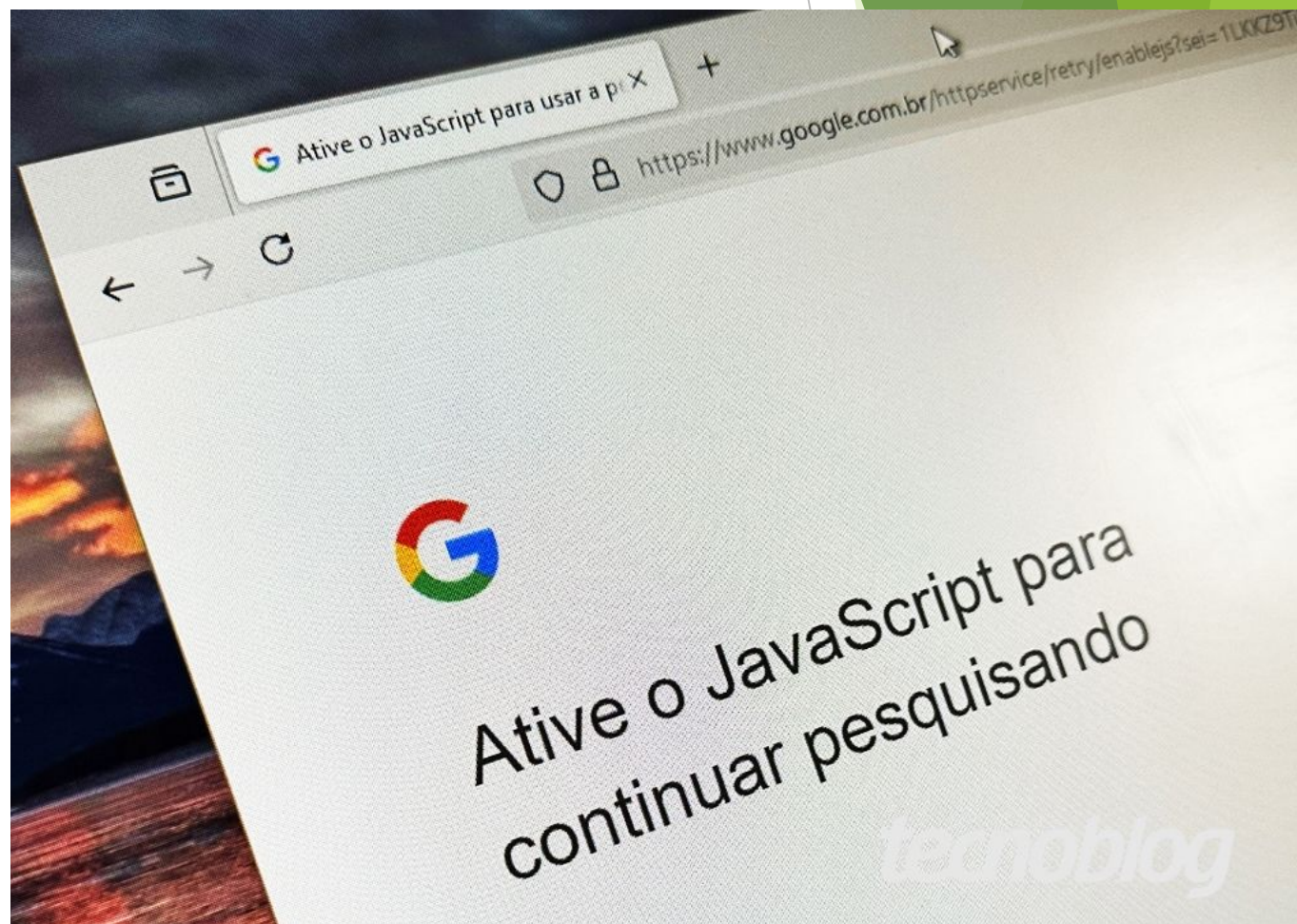
Client Side



JS: linguagens client-side

Linguagens que rodam do lado do cliente

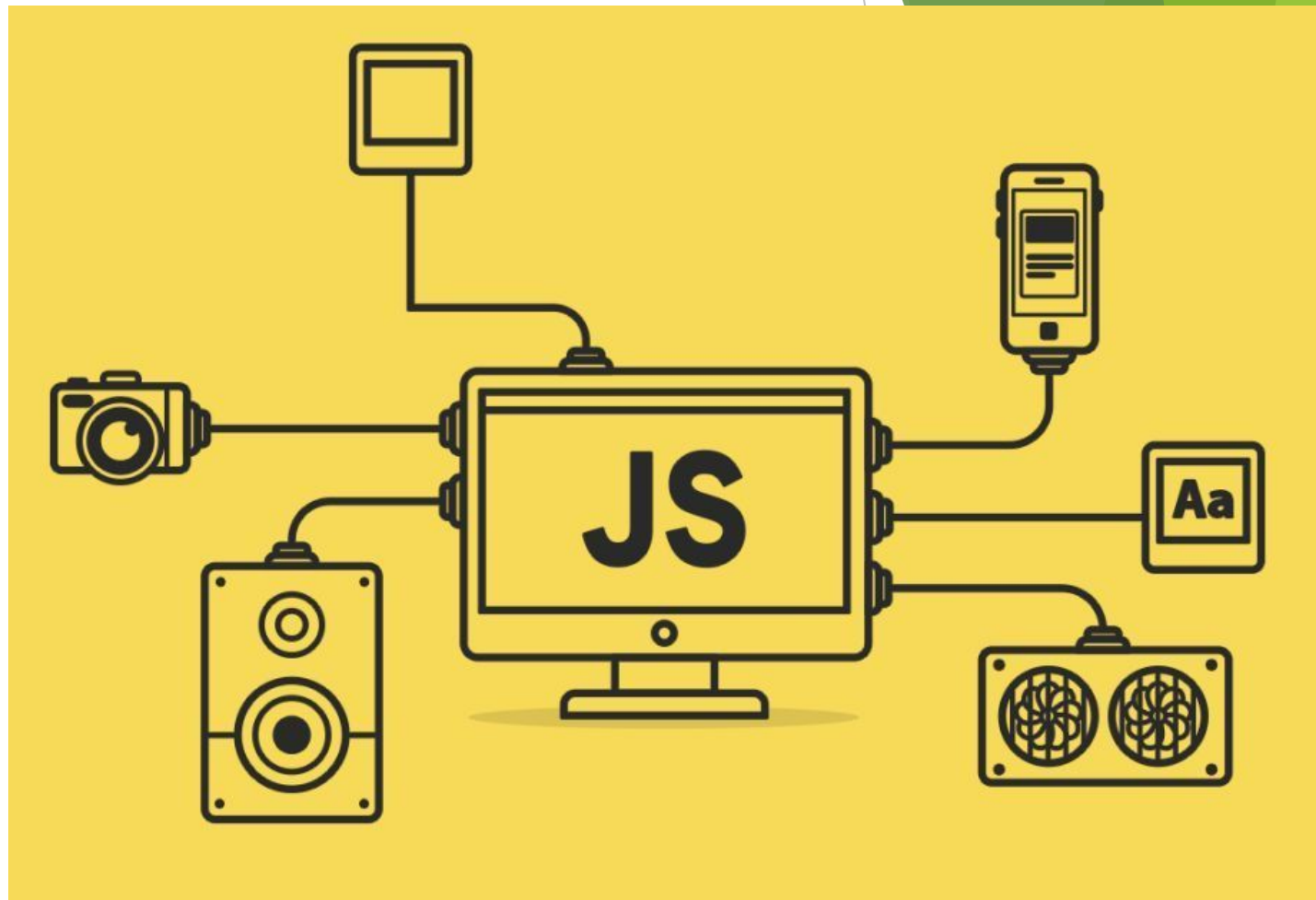
- ❖ São **executadas** utilizando apenas o navegador do usuário.
- ❖ Uma vez os arquivos carregados, não precisa de **novas requisições** no servidor.



Histórico do JavaScript

JS: histórico

- ❖ Nos anos 90, as tecnologias utilizadas para criar páginas era **HTML** para estrutura e **CSS** para controlar a aparência.



JS: histórico

Surgiu do problema de “como deixar as páginas interativas?”

- ❖ Clicar e acontecer algo
- ❖ Validar formulário
- ❖ Atualizar conteúdo sem recarregar a página manualmente

JavaScript



JS: histórico

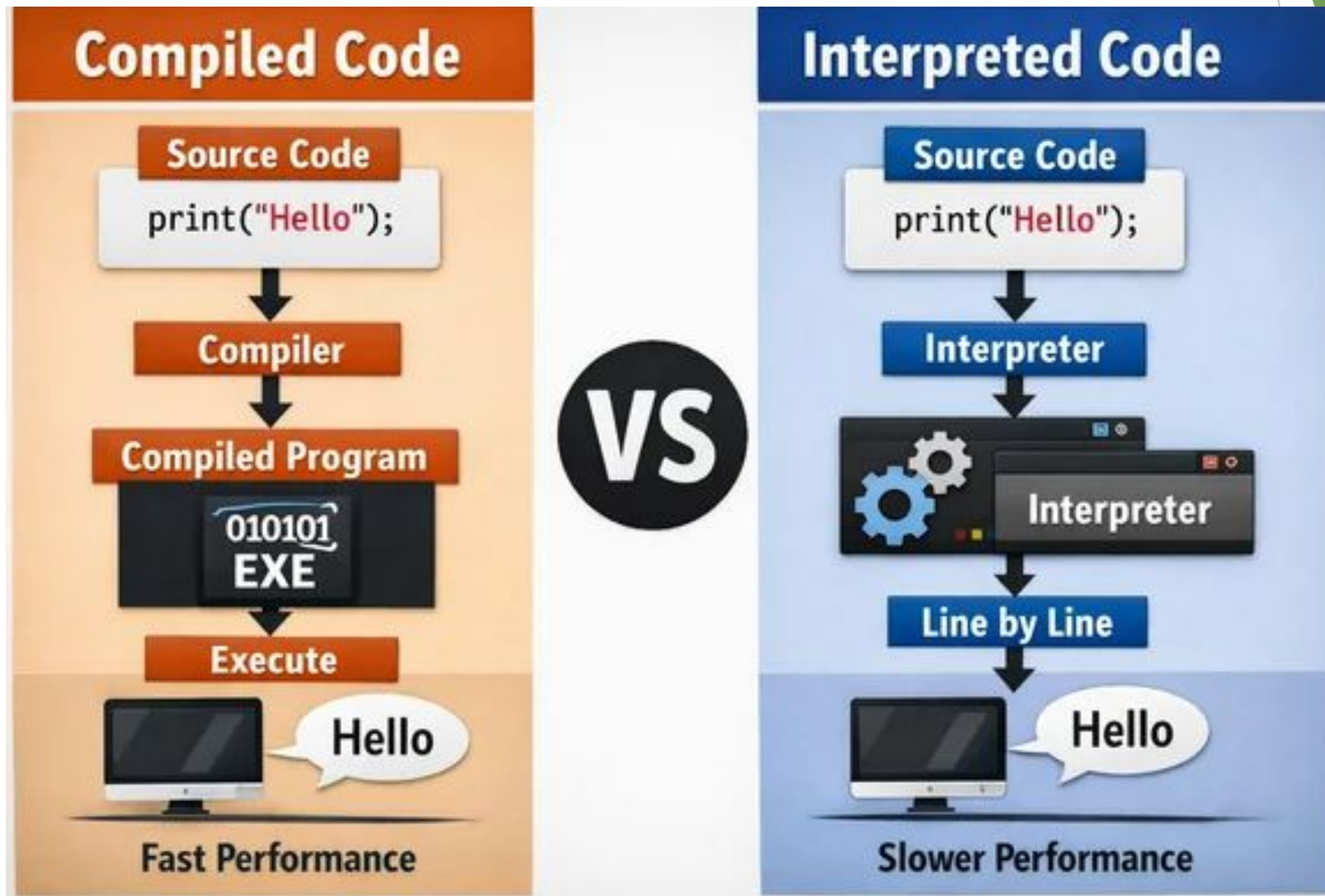
- ❖ Foi criada pelo desenvolvedor Brendan Eich na empresa Netscape em 1995.
- ❖ É uma linguagem de programação interpretada que é **executada** diretamente no navegador.



JS: Interpretada x Compilada

- ❖ **Interpretadas:** Traduzem e executam linha por linha durante a execução do programa.
- ❖ **Compiladas:** Utilizam um compilador para **traduzir** todo o código fonte para linguagem de máquina de uma vez antes de ser **executado**, gerando um arquivo executável separado.

JS: Compilada x Interpretada



JS: histórico

O JS trocou de nome algumas vezes em 1995:

- ❖ Mocha (criação)
- ❖ LiveScript (desenvolvimento)
- ❖ depois virou JavaScript (por marketing, por causa do Java)
- ❖ ECMAScript (1997, após padronização)

Livescribe™

JS: histórico

- ❖ Com a popularização do navegador **Netscape**, várias empresas começaram a criar seus próprios navegadores.
- ❖ E em 1996 a **Microsoft** lançou o Internet Explorer 3.
- ❖ Mas em vez de usar o JavaScript da Netscape, criou o seu próprio, o JScript.



JS: histórico



Fonte: Winworld, 1997

JS: histórico

- ❖ Causando um grande problema, pois o mesmo código JavaScript não funcionava em diferentes navegadores.
- ❖ Criando a “guerra dos navegadores” (Browser Wars).
- ❖ E por isso, em 1997 a Ecma International criou o padrão **ECMAScript 1 (ES1)**.



JS: histórico

E o resultado disso foi:

- ❖ Internet Explorer, Netscape e outros navegadores passaram a seguir o mesmo padrão.
- ❖ Reduziu inconsistências.
- ❖ Facilitou a vida dos desenvolvedores.



The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern, layered effect. The shapes are positioned on the left and right sides of the frame, leaving a central white area for the text.

EcmaScript

JS: O que é EcmaScript então?

- ❖ É o documento oficial que define como a linguagem deve funcionar.

Ele define:

- ❖ Sintaxe (como escrever código).
- ❖ Regras.
- ❖ Comportamentos.



JS: JavaScript x EcmaScript

- ❖ Desta forma, o JavaScript é uma implementação do EcmaScript.

Ou seja:

- ❖ EcmaScript = regras
- ❖ JavaScript = aplicação prática dessas regras



ES vs **JS**
ECMAScript **JavaScript**

Fonte: Oliveira, 2020

JS: JavaScript x EcmaScript

- ❖ O JavaScript é a linguagem de programação que usamos.
- ❖ EcmaScript é o conjunto de regras que garante que ela funcione igual em todo lugar



JavaScript
A programming
language



ECMAScript 6
A scripting-language
specification

Fonte: Singh, 2025

JS: JavaScript x Java

- ❖ Apesar do nome parecido, a semelhança entre Java e JavaScript é apenas isso mesmo.
- ❖ O JavaScript foi criado em 1995 pela *Netscape Communications Corporation*, com o objetivo de ser uma linguagem leve que rodasse diretamente no navegador para tarefas simples, como validar formulários HTML e executar **pequenas interações** no lado do cliente (client-side).



Fonte: Oliveira, 2020

JS: JavaScript x Java

- ❖ Já o Java foi criado em 1995 pela Sun Microsystems, surgiu para ser uma linguagem mais robusta, voltada para aplicações corporativas, **sistemas maiores** e execução em diferentes plataformas.
- ❖ O nome “JavaScript” foi uma escolha de marketing da época, justamente para **aproveitar a popularidade do Java**, que já estava ganhando destaque no mercado.

JS: JavaScript x Java

JS

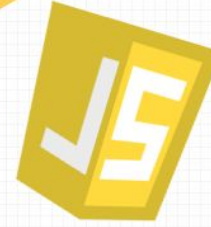
```
<html>
<head>
<title>My First JavaScript
Code!!!</title>
<script>
alert("Hello World!");
</script>
</head>
<body>
</body>
</html>
```



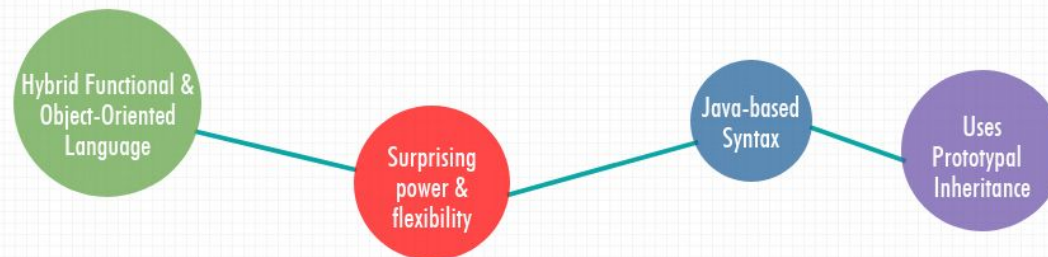
```
class A {
    public static void
    main(String args[])
    {
        System.out.println("Hello
World!");
    }
}
```

JavaScript

A History for Beginners



Defining Characteristics



Advantages

- Can be learned quickly
- Enables developers to build quickly
- Runs quickly
- Has a subset of good parts

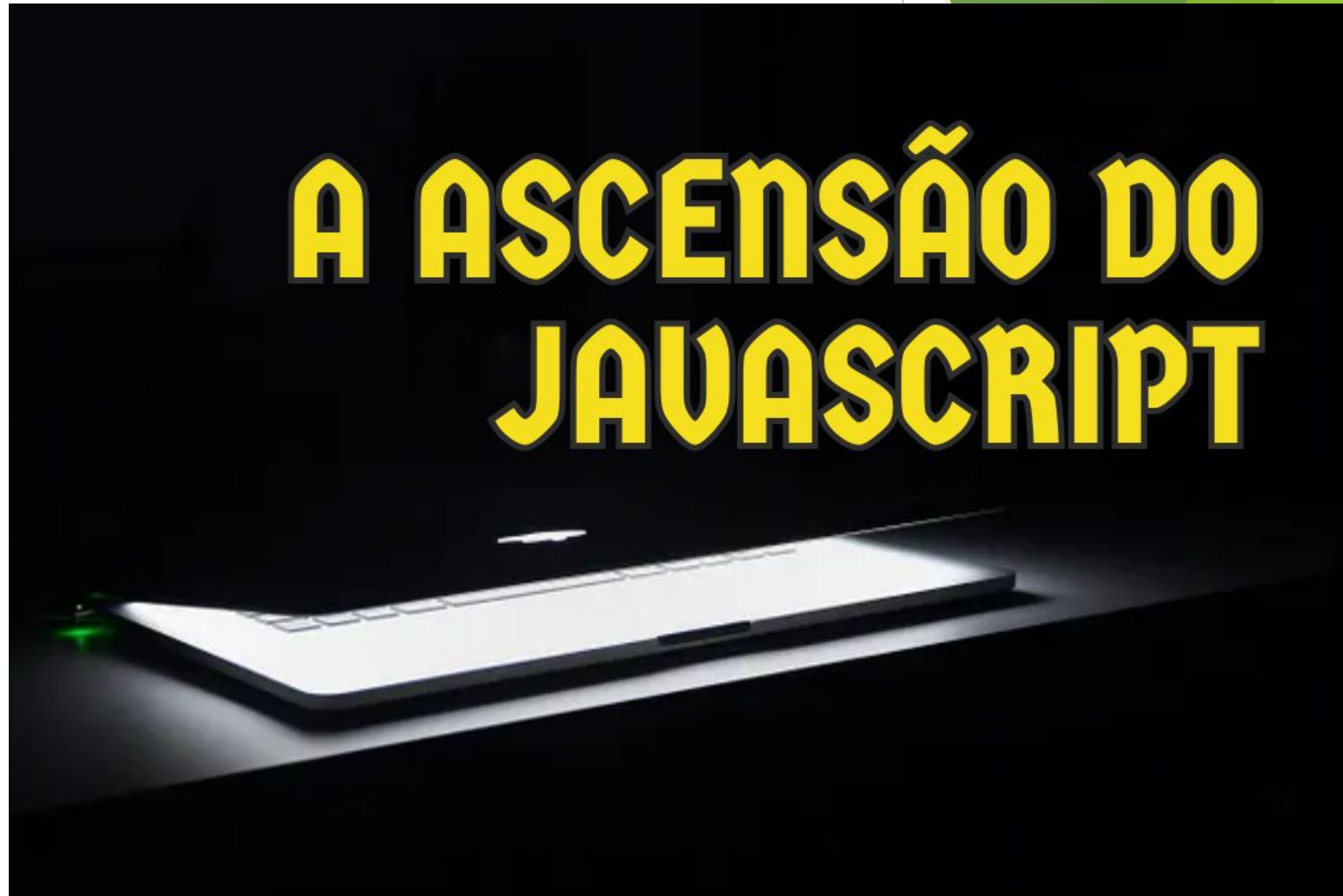
Disadvantages

- Type coercion surprises
- Lack of proper module system
- Requires discipline and training
- No control over lower-level layers

JS: Artigo complementar

Artigo complementar.

- ❖ Clique na imagem ao lado para acessar um artigo complementar ao tema.



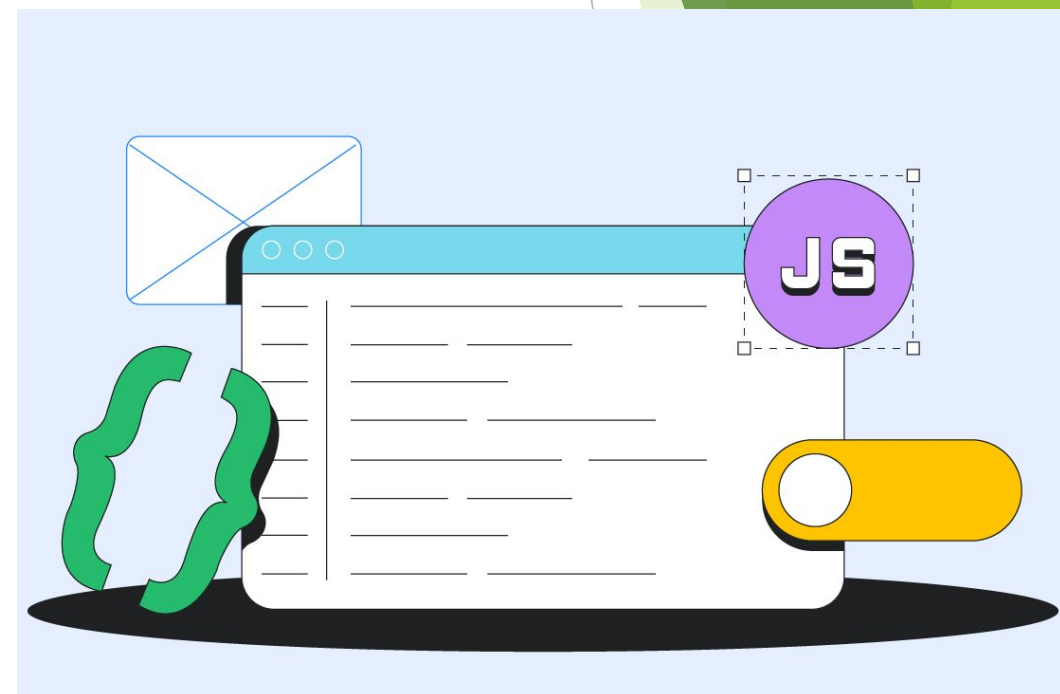
JS: O que é

JS: O que é

- ❖ O JavaScript é uma linguagem de programação criada para tornar páginas web **interativas** e **dinâmicas**.

Sem ele, a web seria basicamente só texto e imagens estáticas (HTML + CSS).

- ❖ HTML: estrutura
- ❖ CSS: aparência
- ❖ JavaScript: comportamento



JS: O que é

É uma das linguagens mais populares da web, e suas maiores **características** são:

- ❖ Tipagem dinâmica
- ❖ Baseada em objetos
- ❖ Orientada a eventos
- ❖ Realiza avaliação em tempo de execução



JS: principais características

JS: Principais características

1. Tipagem dinâmica

- ❖ Significa que você não precisa declarar o **tipo da variável** (string, número, etc.).
- ❖ O JavaScript decide isso automaticamente baseado no **valor atribuído**.

```
let valor = 10;           // número  
valor = "dez";           // agora virou string  
valor = true;            // agora é boolean
```

JS: Principais características

2. Baseada em objetos

- ❖ Trabalha com objetos o tempo todo, mesmo quando você não percebe.
- ❖ Um objeto é um conjunto de propriedades e comportamentos (métodos).

```
let pessoa = {  
  nome: "Ana",  
  idade: 25,  
  falar: function() {  
    console.log("Olá!");  
  }  
};  
  
pessoa.falar(); // "Olá!"
```

JS: Principais características

3. Orientada a eventos

- ❖ Funciona reagindo a ações do usuário (eventos), como cliques, digitação e carregamento no navegador.
- ❖ O **HTML** cria os elementos que disparam os eventos e o **JavaScript** reage a esses eventos.

```
document.getElementById("btn").addEventListener("click", () => {  
  console.log("Botão clicado!");  
});
```

JS: Principais características

4. Avaliação em tempo de execução (runtime)

- ❖ Funções podem ser criadas e executadas dinamicamente.

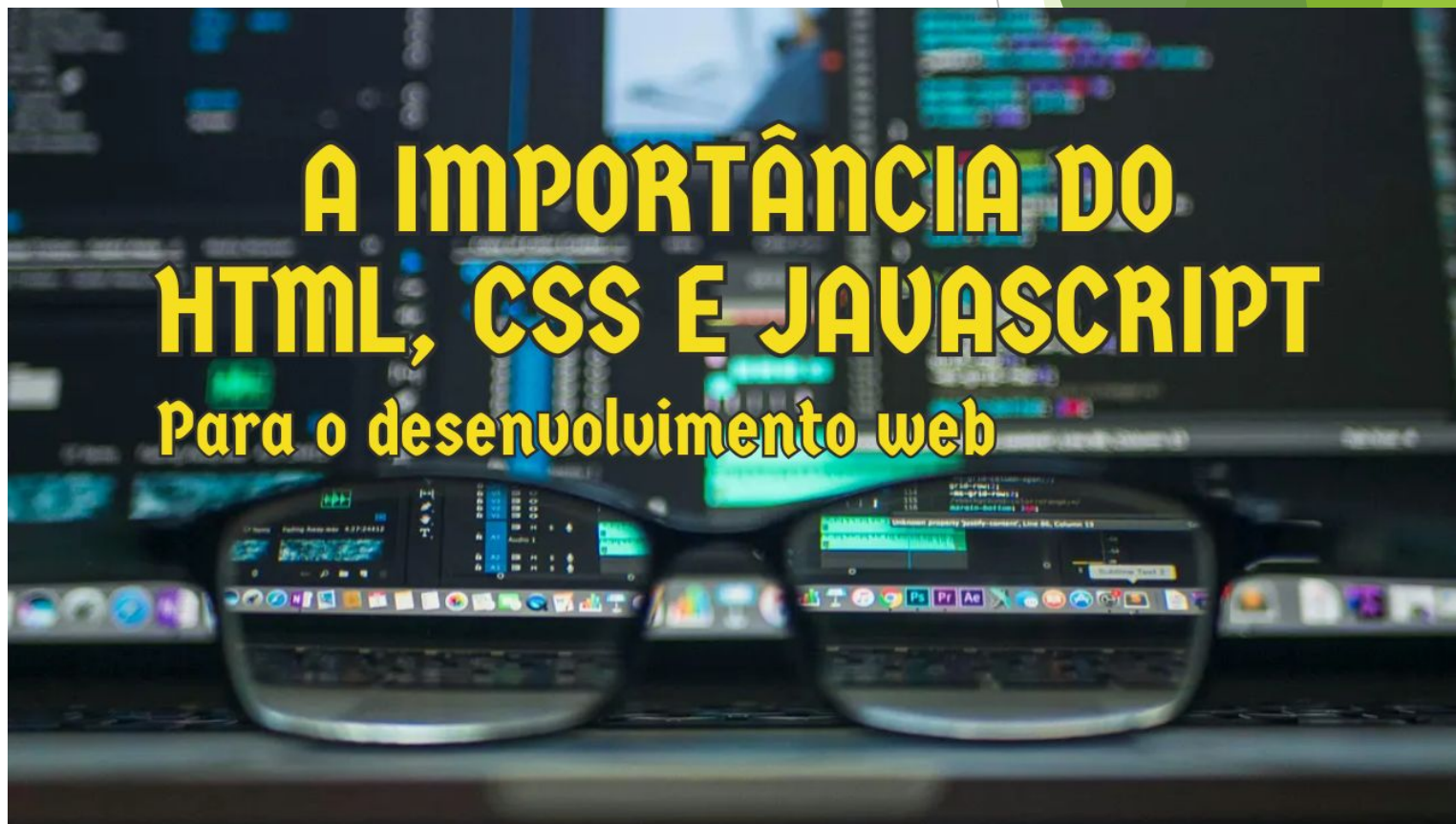
Exemplo: Você cria a função na hora, passa ela como argumento e outra função decide quando executar.

```
function executar(func) {  
  func();  
}  
  
executar(() => console.log("Rodando em tempo real"));
```


JS: Artigo complementar

Artigo complementar.

- ❖ Clique na imagem ao lado para acessar um artigo complementar do tema.



JS: desenvolvimento web

Com JavaScript você consegue:

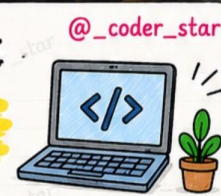
- ❖ reagir a cliques
- ❖ validar formulários
- ❖ atualizar conteúdo sem recarregar a página
- ❖ criar animações
- ❖ consumir dados da internet (APIs)
- ❖ construir aplicações completas (tipo sistemas)



Fonte: sencha, 2023

JS

FEATURES OF JAVASCRIPT



@_coder_star

1 MAIN FEATURES OF JAVASCRIPT

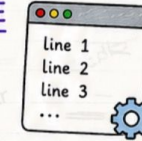
1 LIGHTWEIGHT

- ✓ JavaScript consumes **less** memory and runs quickly.



2 INTERPRETED LANGUAGE

- ✓ Code executes line by line inside **browser**.



3 DYNAMIC TYPING

- ✓ Variable type changes automatically.

JS

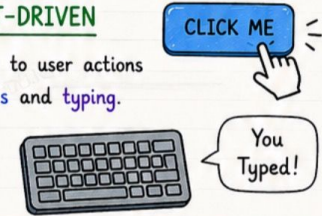
```
let data = 10;  
data = "Hello";
```

Number
10

String
"Hello"

4 EVENT-DRIVEN

- ✓ Responds to user actions like **clicks** and **typing**.



5 PLATFORM INDEPENDENT

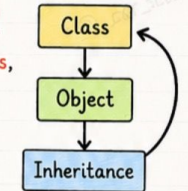
- ✓ Runs on **Windows**, **Linux**, **Mac**, and **browsers**.



Write once, run anywhere!

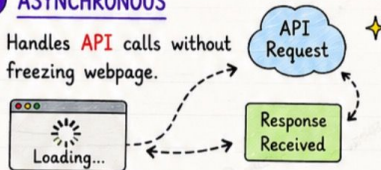
6 OBJECT-ORIENTED

- ✓ Supports **objects**, **classes**, and **inheritance**.



7 ASYNCHRONOUS

- ✓ Handles **API** calls without freezing webpage.

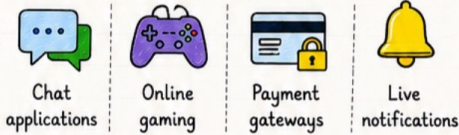


3 ADVANTAGES

- ★ **Fast** development
- ★ Huge **community** support
- ★ Easy to **learn**
- ★ Powerful **ecosystem**

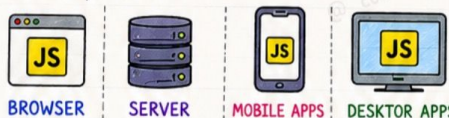


2 REAL-LIFE USAGE



4 BONUS POINT

JavaScript can run:



QUICK REVISION

Fonte: CoderStar, 2025

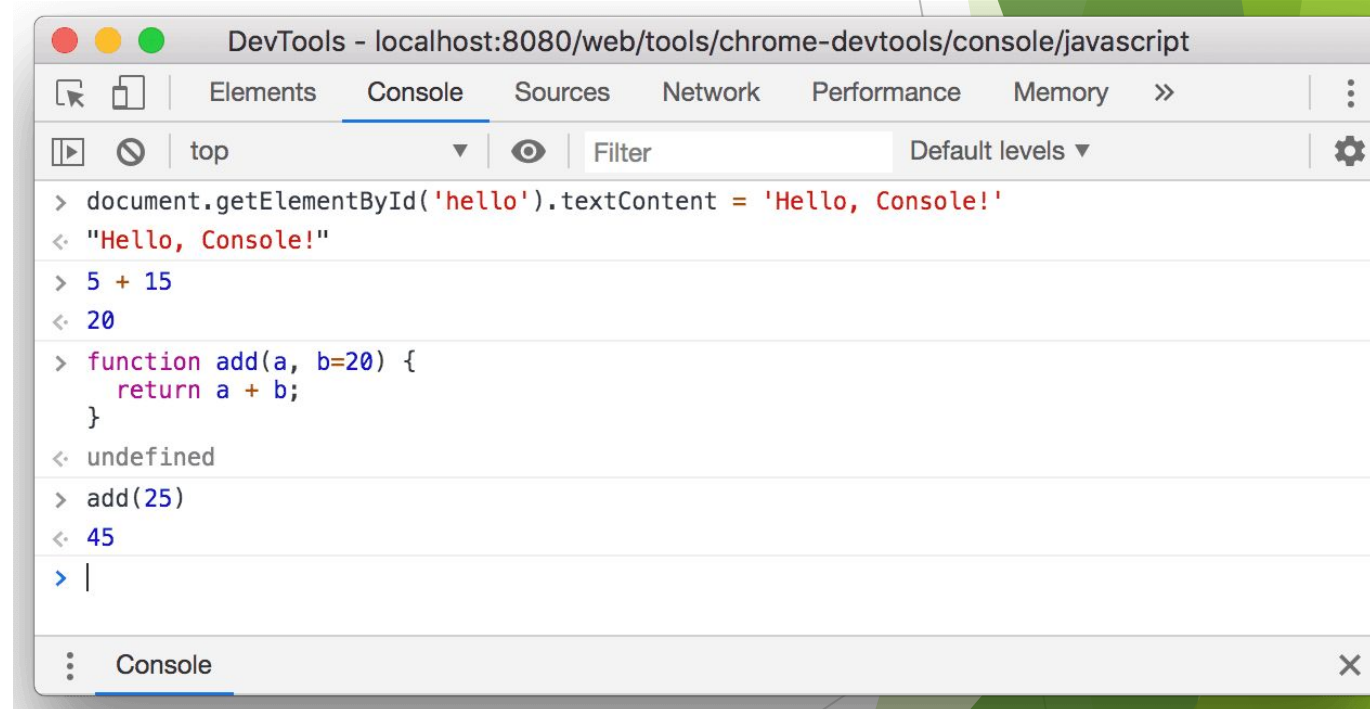
The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

JS: como executar

JS: execução do JavaScript

1. No navegador (client-side)

- ❖ Todo navegador moderno possui um motor que **executa** JavaScript.
- ❖ Aqui o JavaScript interage diretamente com a página (DOM).



Fonte: Basques, 2018

JS: execução do JavaScript

2. No servidor (server-side)

- ❖ Com Node.js, JavaScript também roda fora do navegador.

Isso permite:

- ❖ criar APIs
- ❖ trabalhar com banco de dados
- ❖ construir back-end completo



JS: como incluir em uma página

JS: formas de incluir o JavaScript

Como incluir o JavaScript numa página web, similar ao CSS, temos as seguintes formas:

- ❖ Inline
- ❖ Interno (na página HTML pela tag script)
- ❖ Externo

JS: formas de incluir o JavaScript

❖ Inline:

O JavaScript é colocado diretamente nas **tags** no **HTML**, dentro de atributos como:

❖ *onclick*

❖ *onchange*

❖ *oninput*

```
<button onclick="alert('Olá!')">Clique</button>
```

JS: formas de incluir o JavaScript

❖ Interno:

É adicionado na **tag script** antes de encerrar a tag body, e o atributo *type* indica o tipo do script.

```
<script type="text/javascript">  
    document.write("Parte gerada com JavaScript");  
</script>  
</body>  
</html>
```

JS: formas de incluir o JavaScript

- ❖ Tanto o **inline** quando o **interno** não são as melhores formas de incluir o JS numa página, pois não separa bem as **responsabilidades** do HTML.
- ❖ O arquivo **HTML** deverá ficar responsável pela estruturação e **JS** pela interação. A falta dessa separação dificulta o **reuso** de código.
- ❖ A forma indicada para inclusão de JS em uma página é incluir de um **arquivo externo**.



JS: formas de incluir o JavaScript

❖ Externo:

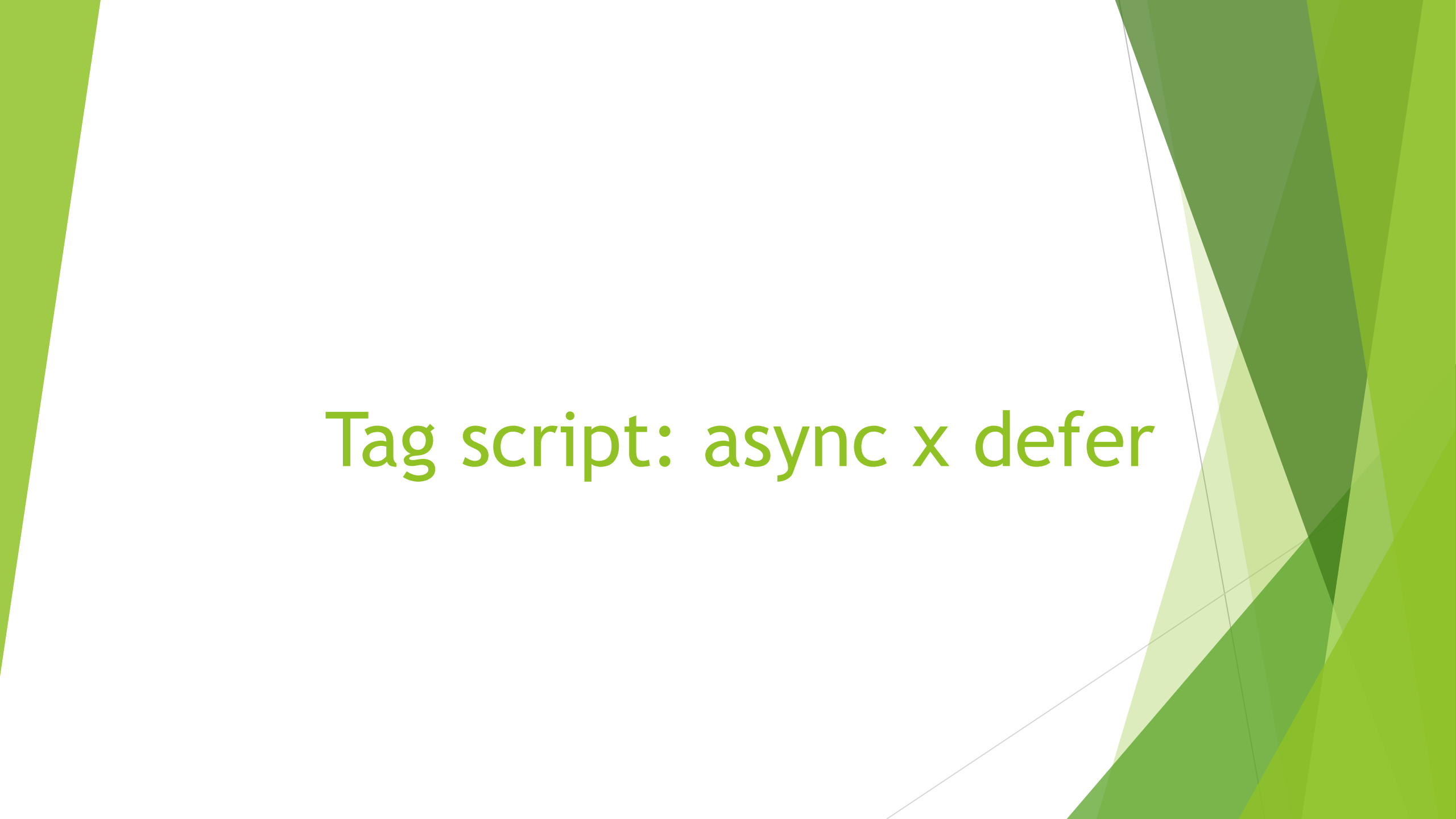
O JavaScript é incluído na página por um link de um arquivo externo, pela tag `<script>` dentro da tag `<head>`.

```
<!doctype html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vendas de TVs e acessórios online</title>
    <link rel="stylesheet" href="./styles.css" />
    <link rel="stylesheet" href="./styles-responsive-flex.css" />
    <script src="script.js"></script>
  </head>
```

JS: formas de incluir o JavaScript

Por que essa é a melhor forma?

- ❖ **Separação de responsabilidades:** o HTML faz a estrutura, CSS cuida da estilizar e JS faz as interações.
- ❖ **Código mais Limpo:** o código não fica misturado em um único arquivo, então é mais fácil de ler e entender e mais organizado.
- ❖ **Reutilizável:** o mesmo JS pode ser usado em outras páginas.
- ❖ **Performático:** o navegador pode armazenar arquivos JS no cache, evitando downloads repetidos e acelerando o carregamento.

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Tag script: async x defer

HTML: <script>

- ❖ Permite incluir script dinâmico que será executado no computador do internauta. Você pode construir as **linhas de código** do script no elemento <script> ou em um documento externo, referenciado por meio do atributo *src*.

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport"
  | | | content="width=device-width, initial-scale=1.0" />
  <link rel="stylesheet" type="text/css" href="styles.css">
  <title>Vendas de TVs e acessórios online</title>
  <script src="script.js"></script>
</head>
```

HTML: <script> e atributo async

- ❖ Essa tag pode conter o atributo *async* que influencia em como o script é lido e executado.
- ❖ Com esse atributo, o arquivo js é baixado sem pausar o interpretador (em paralelo) e executa na primeira oportunidade após carregado.

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport"
    | | content="width=device-width, initial-scale=1.0" />
  <link rel="stylesheet" type="text/css" href="styles.css">
  <title>Vendas de TVs e acessórios online</title>
  <script src="script.js" async></script>
</head>
```

HTML: <script> e atributo defer

- ❖ O atributo *defer* na tag <script> indica que o script vai ser baixado em paralelo (como o async) mais que sua execução vai acontecer somente quando a página terminar de ser interpretada (parsing).

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0" />
  <link rel="stylesheet" type="text/css" href="styles.css">
  <title>Vendas de TVs e acessórios online</title>
  <script src="script.js" defer</script>
</head>
```

HTML: <script> async x defer

- ❖ Tanto o *async* como o *defer* existem para evitar que o navegador **pause a renderização** da página para baixar e renderizar o arquivo JavaScript.

Sem seu uso, o processo de renderização acontece assim:

- ❖ O navegador lê o HTML.
- ❖ Encontra o <script>.
- ❖ Para de processar o HTML.
- ❖ Baixa o JavaScript e o Executa.
- ❖ Volta e continua lendo o HTML.

Isso pode deixar a página mais lenta para carregar.

HTML: <script> async x defer

- ❖ Utilizar o *defer* evita o erro “Cannot read properties of null”, que acontece quando um script JavaScript busca manipular algum elemento na página que ainda não foi renderizado, pois o arquivo JS teve sua execução antes da renderização da página ser finalizada.

```
<head>
|  <script src="app.js" async></script>
</head>
<body>
|  <main>
|  |  <section>
|  |  |  <div id="mensagem"></div>
|  |  </section>
|  </main>
</body>

document.getElementById("mensagem").innerText = "Olá";
```


HTML: <script> async x defer

Regra prática

- ❖ Script da aplicação: usar defer
- ❖ Script de terceiros independente: usar async

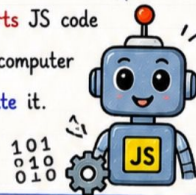
Evite scripts sem nenhum dos dois, exceto quando realmente precisar bloquear a renderização.

JS: Como funciona

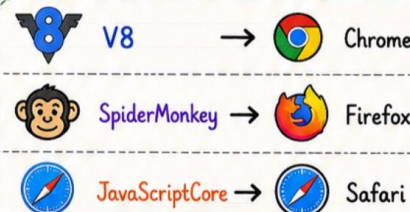
JS JAVASCRIPT ENGINES & HOW JS WORKS

1 WHAT IS JAVASCRIPT ENGINE?

A JavaScript engine **converts** JS code into **machine code** so the computer can **understand** and **execute** it.



2 POPULAR JAVASCRIPT ENGINES



3 WORKING PROCESS

1 PARSING

- Engine reads code syntax.



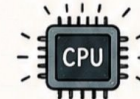
2 COMPILATION

- Converts JS into **optimized machine code**.



3 EXECUTION

- CPU executes instructions.



RESULT

- Task Completed!



4 EXAMPLE

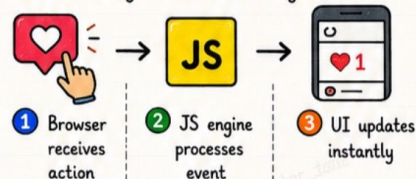
```
let x = 5 + 10;  
console.log(x);  
// Output: 15
```

The engine:

- ☒ Reads code
- ☒ Compiles it
- ☒ Executes output

5 REAL-LIFE EXAMPLE

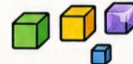
When clicking "Like" on Instagram:



6 IMPORTANT COMPONENTS

MEMORY HEAP

Stores data and objects in memory.

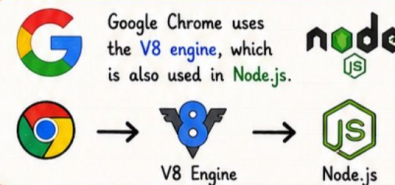


CALL STACK

Keeps track of function calls.

func3()
func2()
func1() → Last In First Out (LIFO)

7 INTERVIEW POINT



- ☒ JS Engine converts code into machine code.
- ☒ It reads, compiles and executes the code.

Remember!

JS: conceitos

Formas de exibir informações

JS: formas de exibir informações

document.write:

- ❖ Permite que uma informação seja criada ou alterada dentro da página HTML, no corpo da página ou em uma tag.
- ❖ Entretanto esse método é considerado obsoleto (*deprecated*), e seu uso atualmente é **desencorajado**.
- ❖ A manipulação do conteúdo da página é realizada por meio do **DOM**, utilizando métodos como *textContent*, *innerHTML* e *appendChild*, que oferecem maior segurança, flexibilidade e desempenho.

```
document.write("<h1>Olá Mundo!</h1>");
```


JS: formas de exibir informações

Janelas de diálogo: existem 3 tipos:

- ❖ Janelas de alerta (alert)
- ❖ Janelas de confirmação (confirm)
- ❖ Janela de entradas de dados (prompt)

JS: formas de exibir informações

❖ Janela de alerta (alert)



```
alert("Atenção!");
```

Fonte: autora, 2026

JS: formas de exibir informações

- ❖ Janela de confirmação (confirm)



```
confirm("Deseja continuar?");
```

Fonte: autora, 2026

JS: formas de exibir informações

- ❖ Janela de entrada de dados (prompt)

A screenshot of a JavaScript prompt dialog box. The dialog has a dark gray background. At the top, the text "Qual é o seu nome?" is displayed in a light gray font. Below the text is a white rectangular input field with rounded corners and a thin white border. At the bottom right of the dialog are two buttons: "OK" and "Cancelar", both with rounded corners and a light gray background.

```
let nome = prompt("Qual é o seu nome?");
```

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Dúvidas?

Exercício

JS: exercício 1 - objetivo

- ❖ Utilize a janelas de diálogo prompt para coletar **dois números** do usuário e fazer a **soma** deles.
- ❖ Exiba o resultado com a janela de diálogo confirm.

Digite o primeiro número:

Digite o segundo número:

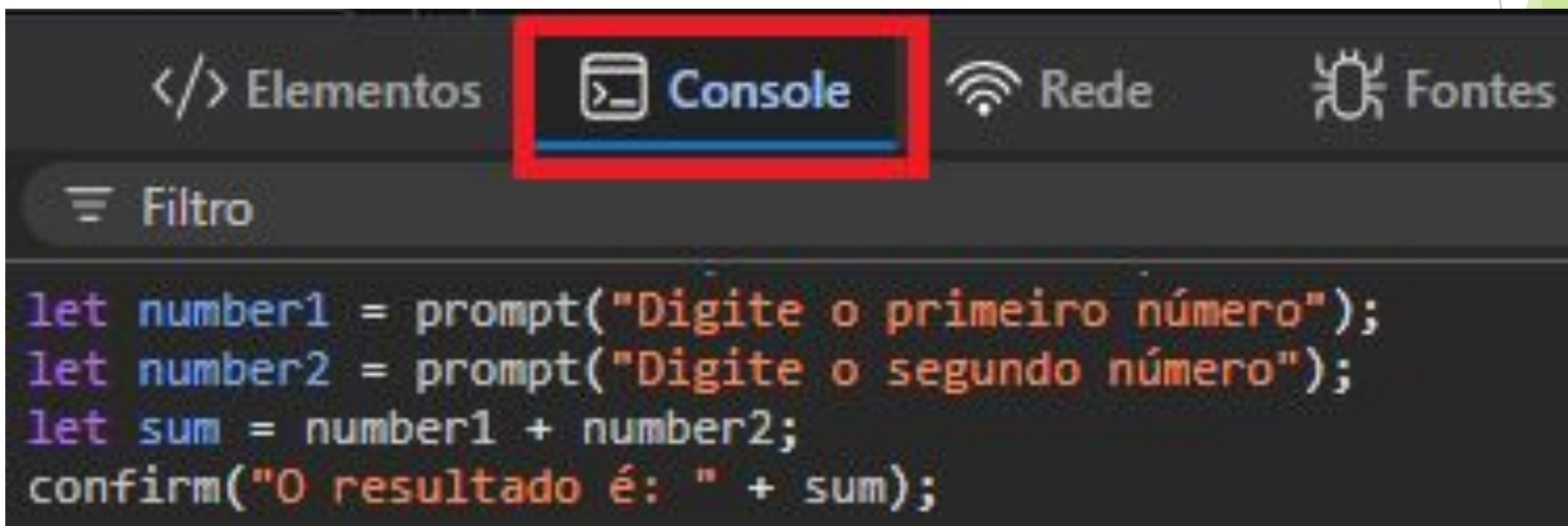
OK Cancelar

O resultado é: 2

OK Cancelar

JS: exercício 1 (código com erro)

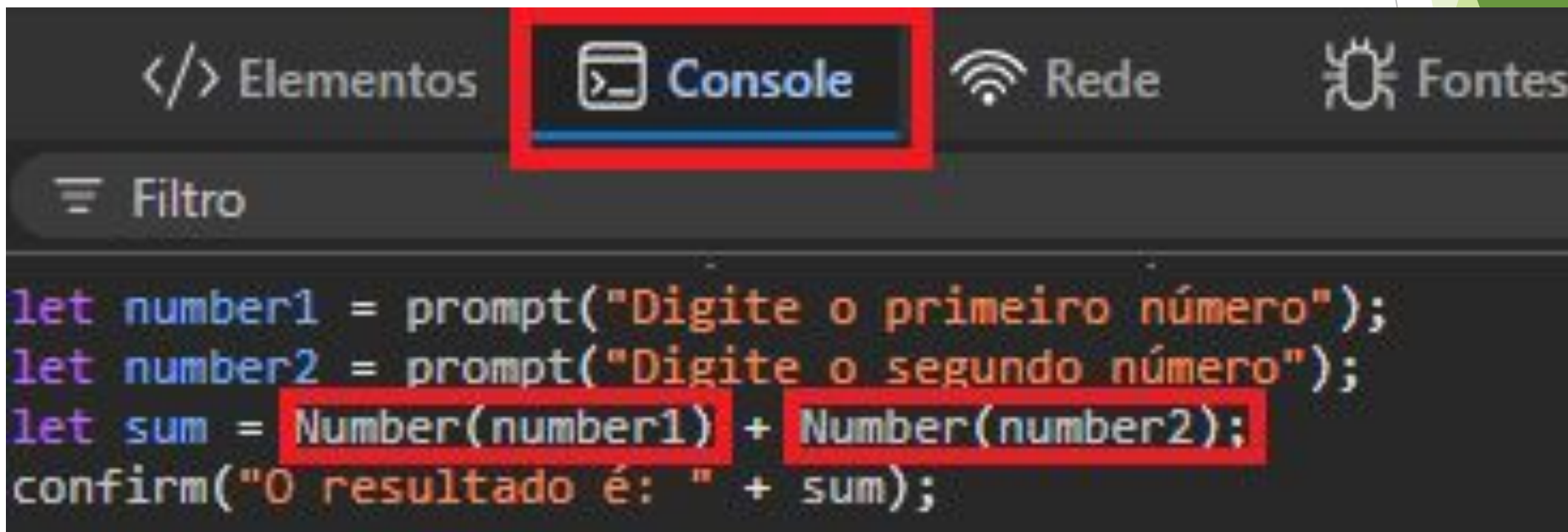
- ❖ Abra as ferramentas de desenvolvedor no seu navegador em qualquer site e vá na aba **console**.
- ❖ Digite o código abaixo no console e pressione **enter**.



```
let number1 = prompt("Digite o primeiro número");  
let number2 = prompt("Digite o segundo número");  
let sum = number1 + number2;  
confirm("O resultado é: " + sum);
```

JS: exercício 2 (código corrigido)

- ✦ Agora teste a nova versão do código abaixo que está corrigido com a **conversão para números** dos valores capturados pelas janelas de prompt, e portanto faz a somatória correta.

A screenshot of a web browser's developer console. The top bar shows four tabs: 'Elementos' (Elements), 'Console' (highlighted with a red box), 'Rede' (Network), and 'Fontes' (Sources). Below the tabs is a search bar labeled 'Filtro'. The console displays the following JavaScript code:

```
let number1 = prompt("Digite o primeiro número");  
let number2 = prompt("Digite o segundo número");  
let sum = Number(number1) + Number(number2);  
confirm("O resultado é: " + sum);
```

The expressions `Number(number1)` and `Number(number2)` in the third line are highlighted with red boxes.

Exercício

- ❖ Acesse o **arquivo de exemplo** no link abaixo.
- ❖ Copie cada bloco de código de exemplo, uma a uma na aba console das **ferramentas de desenvolvedor** do seu navegador, e analise os resultados.
- ❖ Exemplos

Referência da aula

- ❖ LEBEC, Gabriel. JavaScript: a history for beginners. Course Report, 2019. Disponível em: <https://www.coursereport.com/blog/history-of-javascript>. Acesso em: 23 mar. 2026.
- ❖ MILETTO, Evandro M.; BERTAGNOLLI, Silvia C. Desenvolvimento de software II: introdução ao desenvolvimento web com HTML, CSS, javascript e PHP. (Tekne). Porto Alegre: Bookman, 2014. E-book. p.119. ISBN 9788582601969. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9788582601969/>. Acesso em: 11 mai. 2026.
- ❖ TASK INTERNET. O que é HTTPS?. Task Internet, [s.d.]. Disponível em: <https://www.task.com.br/blog/o-que-e-https/>. Acesso em: 30 mar. 2026.

Referência da aula

- ❖ GORDON, Adam. Browser Wars Episode II: The Wavering of a Titan. Medium, 25 abr. 2019. Disponível em: <https://medium.com/@adamjgordon24/browser-wars-episode-ii-the-wavering-of-a-titan-851dc57922fd>. Acesso em: 30 mar. 2026
- ❖ OLIVEIRA, Eugênio P. Conhecendo um pouco do JavaScript. Medium, 8 jun. 2020. Disponível em: <https://medium.com/@eugenio.p.oliveira/conhecendo-um-pouco-do-javascript-cc252354415b>. Acesso em: 30 mar. 2026.
- ❖ FREECODECAMP. Python vs JavaScript: qual a diferença entre essas duas linguagens de programação tão populares. freeCodeCamp, [s.d.]. Disponível em: <https://www.freecodecamp.org/portuguese/news/python-x-javascript-qual-a-diferenca-entre-essas-duas-linguagens-de-programacao-tao-populares/>. Acesso em: 30 mar. 2026.

Referência da aula

- ❖ GOOGLE. JavaScript. Web.dev, 2024. Disponível em: <https://web.dev/javascript?hl=pt-br>. Acesso em: 30 mar. 2026.



univille.br